

Trabalho I - Máquina de Estados

1. Visão do Cenário:

Em nosso cenário, temos duas pessoas trabalhando juntas na criação de um jogo, um programador e um artista.

O programador é responsável por estruturar o código e implementar os assets, teoricamente, fazendo o jogo funcionar. Já o artista tem o papel de apenas criar os assets e passar para o programador.

2. Visão de cada Agente:

Visão do Programador:

O programador deve implementar as artes e criar o código, portanto ele tem o seguinte processo: prototipar, e em sequência programar de fato, que também inclui a implementação de assets. Ao fazer isso ele gera bugs, que devem ser corrigidos posteriormente. O programador só consegue passar para a etapa de programação assim que tem um mínimo de prototipação feita e um número necessário de assets.

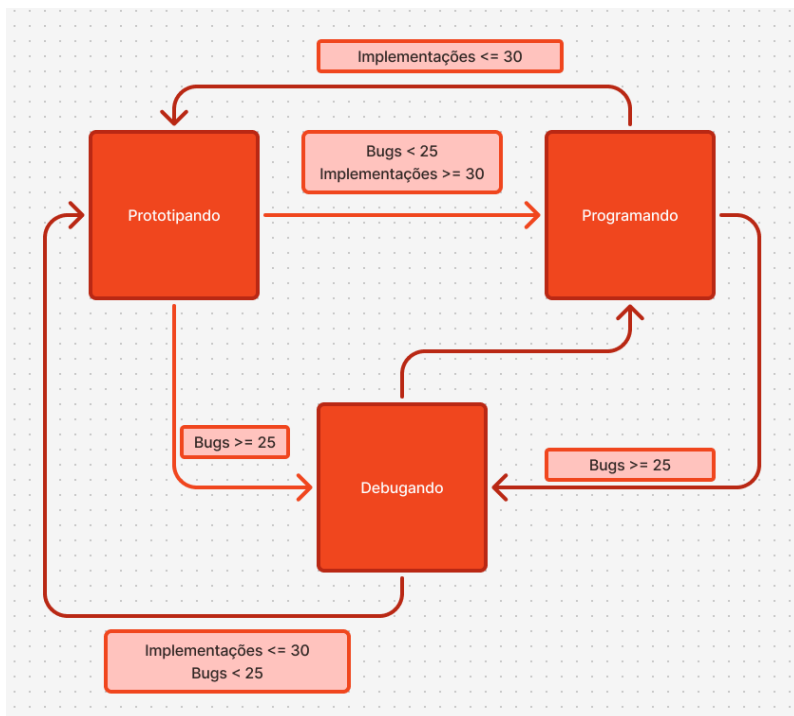
Visão do Artista:

O artista tem o papel de apenas produzir os assets, que neste caso são 2D. Ele começa pesquisando referências, e ao ter um certo nível de referências, ele começa a desenhar, ou seja, produzir assets. Quando o artista tem um número específico de assets, ele os encaminha para o programador e volta a pesquisar referências.

3. Diagrama de Estados:

Programador:





Artista:

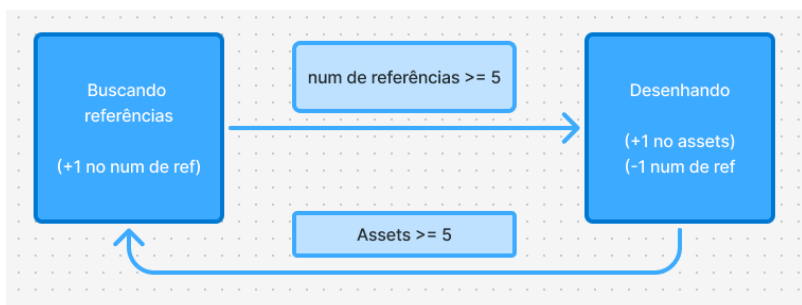
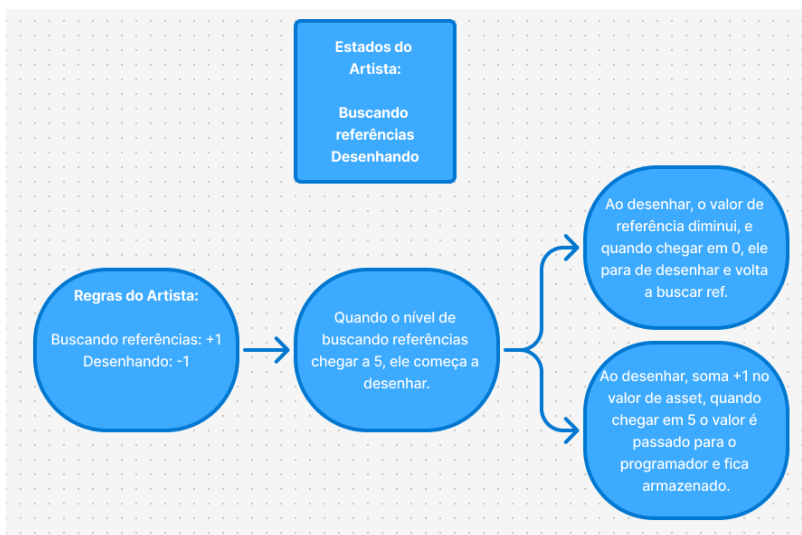
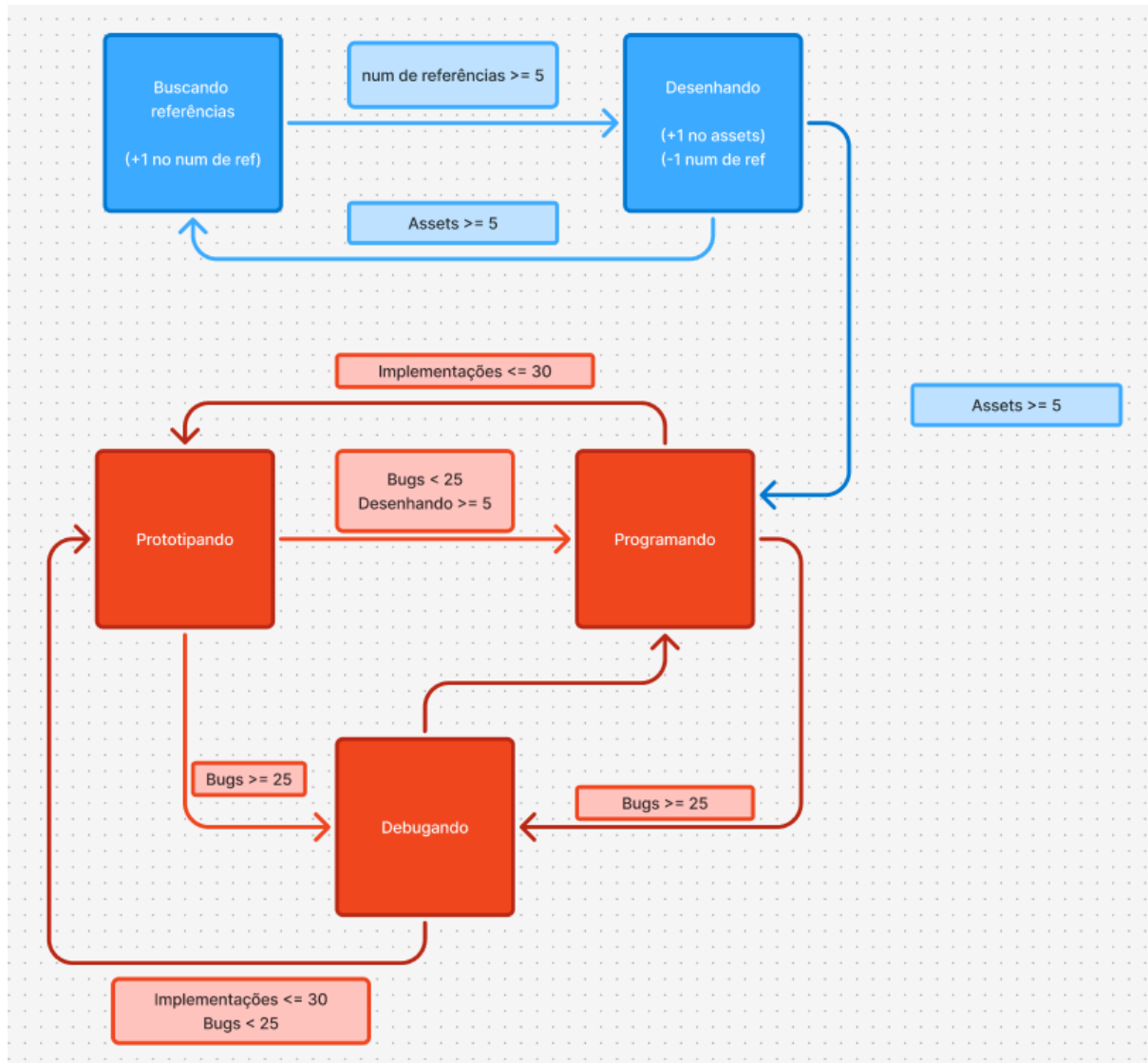


Diagrama Completo:



4. Tabela de Regras:

Programador:

Prototipando	+ 5 implementações + 3 debugs
Programando	+ 5 bugs - 3 implementações - 1 asset
Debugando	- 5 implementações - 10 bugs

Artista:

Buscando referências	+ 1 inspiração
Desenhando	+ 1 asset - 1 inspiração

5. Descrição das Variáveis:

Programador:

- implementações:
 - int
 - intervalo: 32 bits
 - valor inicial = 0
 - limiares ≤ 30
- bugs:
 - int
 - intervalo: 32 bits
 - valor inicial = 0
 - limiares < 25 (bugs) + ≥ 5 (assets)

Artista:

- references:
 - int
 - 32 bits
 - valor inicial = 0
 - limiares = 5
- assets:
 - int
 - 32 bits
 - valor inicial = 0
 - limiares = 5

6. Estrutura do código:

Classes do programador:

- Prototipando:
 - Aumenta em 5 o valor da variável implementações, e aumenta em 3 a variável debugs.
 - Pré requisito para ir programando.
- Programando:
 - Aumenta em 5 o valor da variável bugs, diminui 3 da variável bugs, e diminui 1 da variável asstes.
- Debugando:
 - Subtrai 10 do valor de debugs e subtrai 5 do valor de implementações.

Classes do artista:

- Inspiração:
 - Adiciona 1 no valor da variável references.
 - Pré requisito para desenhando.
- Desenhando:
 - Adiciona 1 no valor da variável assets e subtrai 1 no valor de referências.
 - Pré requisito para programar.

7. Resultados esperados:

Ambos os agentes trabalham separadamente até que os dois possam passar para o “programando”.

O programador prototipa, corrige debugs, prototipa novamente e finalmente pode passar para programando, já que ele tem implementações e assets suficientes e poucos debugs.

Enquanto isso o artista continua pesquisando e desenhando, e passando os assets, que ficam armazenados, esperando o programador ter implementações o bastante e poucos debugs.

8. Limitações:

Acreditamos que não tenha nada que faltou ser implementado para o programa funcionar.

Provavelmente o que faria o programa ficar melhor e mais completo, seria o balanceamento dos valores e a inclusão de outros agentes com outras funções, tornando o processo de criação do jogo mais real.

9. Referências:

Para a ideia do projeto, utilizamos apenas nossas experiências como estudantes de jogos digitais. Já para o trabalho, utilizamos a referência da própria professora e ajuda externa de IA (Gemini e ChatGPT).